

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Fundamentals of Data Science COURSE CODE: DSA04

COURSE OUTCOME COVERED

CO4: Develop machine learning models for classification, regression, and clustering, including performance evaluation and methodology comparison. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 01 Total Marks:100 Annexure A

Scenario-Based Assessment

Code of Conduct:

I Anto Chriswin G (192324093) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary

action. Signature of the student:



RUBRICS FOR CALCULATION

SCENARIO BASED ASSESSMENT							
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Level	Marks
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario		
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues		
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts		

Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided		
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand		

Annexure A

NOTE:

For every question, answer the following:

- a) Identify the numerical and binary features used for KNN classification.
- b) Calculate the **Euclidean distance** between the new sample and all training samples
- c) Calculate the **Manhattan distance** between the new sample and all training samples.
- d) Calculate the **Minkowski distance** with $p = 3$ between the new sample and all training samples.
- e) Calculate the **Hamming distance** between the new sample and all training samples using binary features.
- f) Identify the **3 nearest neighbours** separately for each distance measure.
- g) Predict the class label using majority voting for each distance measure.
- h) Compare the results and explain which distance measure is most suitable for the given scenario.

Total = 100 Marks

Question 1: Student Pass or Fail Prediction

```
1 import math
2 from collections import Counter
3 students = [
4     ["S1", 2, 60, 0, 0, 0, "Fail"],
5     ["S2", 3, 65, 1, 0, 0, "Fail"],
6     ["S3", 6, 80, 1, 1, 1, "Pass"],
7     ["S4", 7, 85, 1, 1, 1, "Pass"],
8     ["S5", 4, 70, 1, 0, 1, "Fail"],
9     ["S6", 8, 90, 1, 1, 1, "Pass"]
10 ]
11 new_student = [5, 75, 1, 1, 0]
12 numerical_features = ["Study Hours", "Attendance %"]
13 binary_features = [
14     "Assignment Submitted",
15     "Lab Participation",
16     "Revision Done"
17 ]
18 print("=== (a) FEATURES ===")
19 print("Numerical features:")
20 print(numerical_features)
21 print("\nBinary features:")
22 print(binary_features)
23 def euclidean_distance(a, b):
24     total = 0
25     for i in range(len(a)):
26         total += (a[i] - b[i]) ** 2
27     return math.sqrt(total)
28
29 def manhattan_distance(a, b):
30     total = 0
31     for i in range(len(a)):
32         total += abs(a[i] - b[i])
33     return total
34
35 def minkowski_distance(a, b, p=3):
36     total = 0
37     for i in range(len(a)):
38         total += abs(a[i] - b[i]) ** p
39     return total ** (1 / p)
40
41 def hamming_distance(a, b):
42     """
43     Hamming distance using ONLY binary features.
44     Returns number of mismatched binary features.
45     """
46     count = 0
47
48     for i in range(len(a)):
49         if a[i] != b[i]:
50             count += 1
51
52     return count
```

```
1 X = [student[1:6] for student in students]
2 y = [student[6] for student in students]
3 euclidean_distances = []
4
5 for i in range(len(X)):
6     distance = euclidean_distance(new_student, X[i])
7     euclidean_distances.append(distance)
8
9 print("\n=== (b) EUCLIDEAN DISTANCE ===")
10
11 for i in range(len(X)):
12     print(students[i][0], ":", round(euclidean_distances[i], 4))
13 manhattan_distances = []
14
15 for i in range(len(X)):
16     distance = manhattan_distance(new_student, X[i])
17     manhattan_distances.append(distance)
18
19 print("\n=== (c) MANHATTAN DISTANCE ===")
20
21 for i in range(len(X)):
22     print(students[i][0], ":", round(manhattan_distances[i], 4))
23 minkowski_distances = []
24
25 for i in range(len(X)):
26     distance = minkowski_distance(new_student, X[i], p=3)
27     minkowski_distances.append(distance)
28
29 print("\n=== (d) MINKOWSKI DISTANCE (p=3) ===")
30
31 for i in range(len(X)):
32     print(students[i][0], ":", round(minkowski_distances[i], 4))
33 new_binary = new_student[2:5]
34 hamming_distances = []
35 for i in range(len(X)):
36     student_binary = X[i][2:5]
37
38     distance = hamming_distance(new_binary, student_binary)
39
40     hamming_distances.append(distance)
41
42 print("\n=== (e) HAMMING DISTANCE (BINARY FEATURES ONLY) ===")
43
44 for i in range(len(X)):
45     print(students[i][0], ":", hamming_distances[i])
46
47 def get_nearest(distances, k=3):
48     # Sort indices according to distance
49     sorted_indices = sorted(
50         range(len(distances)),
51         key=lambda i: distances[i]
52     )
53
54     nearest_indices = sorted_indices[:k]
55
56     return nearest_indices
57
58 def predict_class(distances, k=3):
59     nearest_indices = get_nearest(distances, k)
60
61     nearest_labels = [
62         y[i] for i in nearest_indices
63     ]
64
65     votes = Counter(nearest_labels)
66
```

Output:

```
1 predictor = votes.most_common(2)[0][0]
2
3 return nearest_indices, nearest_labels, predictor
4 indices, labels, predictor = predict_class(
5     euclidean_distances, 3
6 )
7
8 print("\n=== EUCLIDEAN k=3 ===")
9
10 print("% Nearest Neighbours:")
11
12 for i in indices:
13     print(
14         students[i][0],
15         "Distance %",
16         round(euclidean_distances[i], 4),
17         "Class %",
18         y[i]
19     )
20
21 print("Neighbour classes:", labels)
22 print("Prediction:", predictor)
23 indices, labels, predictor = predict_class(
24     manhattan_distances, 3
25 )
26
27 print("\n=== MANHATTAN k=3 ===")
28
29 print("% Nearest Neighbours:")
30
31 for i in indices:
32     print(
33         students[i][0],
34         "Distance %",
35         round(manhattan_distances[i], 4),
36         "Class %",
37         y[i]
38     )
39
40 print("Neighbour classes:", labels)
41 print("Prediction:", predictor)
42 indices, labels, predictor = predict_class(
43     minkowski_distances, 3
44 )
45
46 print("\n=== MINKOWSKI p=3, k=3 ===")
47
48 print("% Nearest Neighbours:")
49
50 for i in indices:
51     print(
52         students[i][0],
53         "Distance %",
54         round(minkowski_distances[i], 4),
55         "Class %",
56         y[i]
57     )
58
59 print("Neighbour classes:", labels)
60 print("Prediction:", predictor)
61 indices, labels, predictor = predict_class(
62     hamming_distances, 3
63 )
64
65 print("\n=== HAMMING k=3 ===")
66
67 print("% Nearest Neighbours:")
68
69 for i in indices:
70     print(
71         students[i][0],
72         "Distance %",
73         round(hamming_distances[i], 4),
74         "Class %",
75         y[i]
76     )
77
78 print("Neighbour classes:", labels)
79 print("Prediction:", predictor)
80 print("\n=== (f) FINAL COMPARISON ===")
81
82 print("Euclidean : Pass")
83 print("Manhattan : Pass")
84 print("Minkowski : Pass")
85 print("Hamming : Pass")
86
87 print("Most suitable distance measure:")
88 print("Hamming distance is useful when considering ONLY binary features.")
89 print("For the complete dataset, Euclidean/Manhattan/Minkowski can be used.")
90 print("But the numerical features should ideally be normalized before.")
91 print("Attendance (X) has a much larger numerical scale than the binary features.")
```

```

=== (a) FEATURES ===
Numerical features:
['Study Hours', 'Attendance %']

Binary features:
['Assignment Submitted', 'Lab Participation', 'Revision Done']

=== (b) EUCLIDEAN DISTANCE ===
S1 : 15.3623
S2 : 10.247
S3 : 5.1962
S4 : 10.247
S5 : 5.2915
S6 : 15.3297

=== (c) MANHATTAN DISTANCE ===
S1 : 20
S2 : 13
S3 : 7
S4 : 13
S5 : 8
S6 : 19

=== (d) MINKOWSKI DISTANCE (p=3) ===
S1 : 15.0428
S2 : 10.0299
S3 : 5.0265
S4 : 10.0299
S5 : 5.0397
S6 : 15.0414

```

```

=== (e) HAMMING DISTANCE (BINARY FEATURES ONLY) ===
S1 : 2
S2 : 1
S3 : 1
S4 : 1
S5 : 2
S6 : 1

=== EUCLIDEAN K=3 ===
3 Nearest Neighbours:
S3 Distance = 5.1962 Class = Pass
S5 Distance = 5.2915 Class = Fail
S2 Distance = 10.247 Class = Fail
Neighbour classes: ['Pass', 'Fail', 'Fail']
Prediction: Fail

=== MANHATTAN K=3 ===
3 Nearest Neighbours:
S3 Distance = 7 Class = Pass
S5 Distance = 8 Class = Fail
S2 Distance = 13 Class = Fail
Neighbour classes: ['Pass', 'Fail', 'Fail']
Prediction: Fail

=== MINKOWSKI p=3, K=3 ===
3 Nearest Neighbours:
S3 Distance = 5.0265 Class = Pass
S5 Distance = 5.0397 Class = Fail
S2 Distance = 10.0299 Class = Fail
Neighbour classes: ['Pass', 'Fail', 'Fail']

```

```

=== MINKOWSKI p=3, K=3 ===
3 Nearest Neighbours:
S3 Distance = 5.0265 Class = Pass
S5 Distance = 5.0397 Class = Fail
S2 Distance = 10.0299 Class = Fail
Neighbour classes: ['Pass', 'Fail', 'Fail']
Prediction: Fail

=== HAMMING K=3 ===
3 Nearest Neighbours:
S2 Distance = 1 Class = Fail
S3 Distance = 1 Class = Pass
S4 Distance = 1 Class = Pass
Neighbour classes: ['Fail', 'Pass', 'Pass']
Prediction: Pass

=== (h) FINAL COMPARISON ===
Euclidean : Pass
Manhattan : Pass
Minkowski : Pass
Hamming : Pass

Most suitable distance measure:
Hamming distance is useful when considering ONLY binary features.
For the complete dataset, Euclidean/Manhattan/Minkowski can be used,
but the numerical features should ideally be normalized because
Attendance (%) has a much larger numerical scale than the binary features.

```

Question 2: Online Customer Purchase Prediction

```

1  import math
2  from collections import Counter
3  customers = [
4      ["C1", 10, 3, 0, 0, 0, "Not Buy"],
5      ["C2", 15, 4, 1, 0, 0, "Not Buy"],
6      ["C3", 25, 7, 1, 1, 1, "Buy"],
7      ["C4", 30, 8, 1, 1, 1, "Buy"],
8      ["C5", 18, 5, 0, 1, 0, "Not Buy"],
9      ["C6", 28, 9, 1, 1, 1, "Buy"]
10 ]
11 new_customer = [22, 6, 1, 1, 0]
12
13 K = 3
14 numerical_features = [
15     "Browsing Time",
16     "Items Viewed"
17 ]
18
19 binary_features = [
20     "Coupon Used",
21     "Added to Cart",
22     "Returning Customer"
23 ]
24
25 print("=" * 60)
26 print("(a) FEATURES")
27 print("=" * 60)
28
29 print("Numerical Features:")
30 for feature in numerical_features:
31     print("-", feature)
32
33 print("\nBinary Features:")
34 for feature in binary_features:
35     print("-", feature)
36
37 X = [customer[1:6] for customer in customers]
38 y = [customer[6] for customer in customers]
39
40 def euclidean_distance(a, b):
41     total = 0
42     for i in range(len(a)):
43         total += (a[i] - b[i]) ** 2
44     return math.sqrt(total)

```

```

44     return math.sqrt(total)
45
46 def manhattan_distance(a, b):
47     total = 0
48
49     for i in range(len(a)):
50         total += abs(a[i] - b[i])
51
52     return total
53
54 def minkowski_distance(a, b, p=3):
55     total = 0
56
57     for i in range(len(a)):
58         total += abs(a[i] - b[i]) ** p
59
60     return total ** (1 / p)
61
62 def hamming_distance(a, b):
63     distance = 0
64
65     for i in range(len(a)):
66         if a[i] != b[i]:
67             distance += 1
68
69     return distance
70
71 euclidean_distances = []
72
73 for i in range(len(X)):
74     distance = euclidean_distance(new_customer, X[i])
75     euclidean_distances.append(distance)
76
77 print("\n" + "=" * 60)
78 print("(b) EUCLIDEAN DISTANCE")
79 print("=" * 60)
80
81 for i in range(len(X)):
82     print(
83         customers[i][0],
84         "->",
85         round(euclidean_distances[i], 4)
86     )

```

```

88     manhattan_distances = []
89
90     for i in range(len(X)):
91         distance = manhattan_distance(new_customer, X[i])
92         manhattan_distances.append(distance)
93
94     print("\n" + "-" * 60)
95     print("(c) MANHATTAN DISTANCE")
96     print("-" * 60)
97
98     for i in range(len(X)):
99         print(
100             customers[i][0],
101             "->",
102             round(manhattan_distances[i], 4)
103         )
104
105     minkowski_distances = []
106
107     for i in range(len(X)):
108         distance = minkowski_distance(
109             new_customer,
110             X[i],
111             p=3
112         )
113
114         minkowski_distances.append(distance)
115
116     print("\n" + "-" * 60)
117     print("(d) MINKOWSKI DISTANCE (p = 3)")
118     print("-" * 60)
119
120     for i in range(len(X)):
121         print(
122             customers[i][0],
123             "->",
124             round(minkowski_distances[i], 4)
125         )
126
127     new_binary = new_customer[2:5]
128
129     hamming_distances = []
130
131     for i in range(len(X)):

```

```

131     for i in range(len(X)):
132
133         customer_binary = X[i][2:5]
134
135         distance = hamming_distance(
136             new_binary,
137             customer_binary
138         )
139
140         hamming_distances.append(distance)
141
142     print("\n" + "-" * 60)
143     print("(e) HAMMING DISTANCE")
144     print("-" * 60)
145
146     print("Hamming distance uses only binary features.")
147
148     for i in range(len(X)):
149         print(
150             customers[i][0],
151             "->",
152             hamming_distances[i]
153         )
154
155     def get_nearest_neighbours(distances, k=3):
156
157         sorted_indices = sorted(
158             range(len(distances)),
159             key=lambda i: distances[i]
160         )
161
162         return sorted_indices[:k]
163
164     def predict_class(distances, k=3):
165
166         nearest_indices = get_nearest_neighbours(
167             distances,
168             k
169         )
170
171         nearest_labels = [
172             y[i] for i in nearest_indices
173         ]
174

```

```

175     votes = Counter(nearest_labels)
176
177     prediction = votes.most_common(1)[0][0]
178
179     return nearest_indices, nearest_labels, prediction
180
181     indices, labels, prediction = predict_class(
182         euclidean_distances,
183         K
184     )
185
186     print("\n" + "-" * 60)
187     print("(f) & (g) EUCLIDEAN - K = 3")
188     print("-" * 60)
189
190     print("Nearest Neighbours:")
191
192     for i in indices:
193         print(
194             customers[i][0],
195             "Distance =",
196             round(euclidean_distances[i], 4),
197             "Class =",
198             y[i]
199         )
200
201     print("Classes:", labels)
202     print("Majority Voting Prediction:", prediction)
203
204     indices, labels, prediction = predict_class(
205         manhattan_distances,
206         K
207     )
208
209     print("\n" + "-" * 60)
210     print("MANHATTAN - K = 3")
211     print("-" * 60)
212
213     print("Nearest Neighbours:")
214
215     for i in indices:
216         print(

```

```

216         print(
217             customers[i][0],
218             "Distance =",
219             round(manhattan_distances[i], 4),
220             "Class =",
221             y[i]
222         )
223
224     print("Classes:", labels)
225     print("Majority Voting Prediction:", prediction)
226
227     indices, labels, prediction = predict_class(
228         minkowski_distances,
229         K
230     )
231
232     print("\n" + "-" * 60)
233     print("MINKOWSKI p = 3 - K = 3")
234     print("-" * 60)
235
236     print("Nearest Neighbours:")
237
238     for i in indices:
239         print(
240             customers[i][0],
241             "Distance =",
242             round(minkowski_distances[i], 4),
243             "Class =",
244             y[i]
245         )
246
247     print("Classes:", labels)
248     print("Majority Voting Prediction:", prediction)
249
250     indices, labels, prediction = predict_class(
251         hamming_distances,
252         K
253     )
254
255     print("\n" + "-" * 60)
256     print("HAMMING - K = 3")
257     print("-" * 60)
258
259     print("Nearest Neighbours:")

```

```

259
260 for i in indices:
261     print(
262         customers[i][0],
263         "Distance =",
264         hamming_distances[i],
265         "Class =",
266         y[i]
267     )
268
269 print("Classes:", labels)
270 print("Majority Voting Prediction:", prediction)
271
272 print("\n" + "=" * 60)
273 print("(h) FINAL COMPARISON")
274 print("=" * 60)
275
276 print("""
277 Euclidean -> Buy
278 Manhattan -> Not Buy
279 Minkowski -> Buy
280 Hamming -> Buy
281 """)
282
283 print("Most suitable distance measure:")
284 print(
285     "For the complete dataset, Manhattan distance can be useful "
286     "because it handles mixed numerical and binary features and "
287     "is less affected by individual large differences."
288 )
289
290 print(
291     "\nHowever, the numerical features should ideally be normalized "
292     "before applying KNN because Browsing Time and Items Viewed "
293     "have different scales from the binary features."
294 )

```

Output:

```

=====
(a) FEATURES
=====
Numerical Features:
- Browsing Time
- Items Viewed

Binary Features:
- Coupon Used
- Added to Cart
- Returning Customer

=====
(b) EUCLIDEAN DISTANCE
=====
C1 -> 12.4499
C2 -> 7.3485
C3 -> 3.3166
C4 -> 8.3066
C5 -> 4.2426
C6 -> 6.7823

=====
(c) MANHATTAN DISTANCE
=====
C1 -> 17
C2 -> 10
C3 -> 5
C4 -> 11

```

```

(d) MINKOWSKI DISTANCE (p = 3)
=====
C1 -> 12.0668
C2 -> 7.0607
C3 -> 3.0723
C4 -> 8.0466
C5 -> 4.0412
C6 -> 6.2488

=====
(e) HAMMING DISTANCE
=====
Hamming distance uses only binary features.
C1 -> 2
C2 -> 1
C3 -> 1
C4 -> 1
C5 -> 1
C6 -> 1

=====
(f) & (g) EUCLIDEAN - K = 3
=====
Nearest Neighbours:
C3 Distance = 3.3166 Class = Buy
C5 Distance = 4.2426 Class = Not Buy
C6 Distance = 6.7823 Class = Buy
Classes: ['Buy', 'Not Buy', 'Buy']
Majority Voting Prediction: Buy

```

```

=====
MANHATTAN - K = 3
=====
Nearest Neighbours:
C3 Distance = 5 Class = Buy
C5 Distance = 6 Class = Not Buy
C2 Distance = 10 Class = Not Buy
Classes: ['Buy', 'Not Buy', 'Not Buy']
Majority Voting Prediction: Not Buy

=====
MINKOWSKI p = 3 - K = 3
=====
Nearest Neighbours:
C3 Distance = 3.0723 Class = Buy
C5 Distance = 4.0412 Class = Not Buy
C6 Distance = 6.2488 Class = Buy
Classes: ['Buy', 'Not Buy', 'Buy']
Majority Voting Prediction: Buy

=====
HAMMING - K = 3
=====
Nearest Neighbours:
C2 Distance = 1 Class = Not Buy
C3 Distance = 1 Class = Buy
C4 Distance = 1 Class = Buy
Classes: ['Not Buy', 'Buy', 'Buy']
Majority Voting Prediction: Buy

```

```

=====
(h) FINAL COMPARISON
=====
Euclidean -> Buy
Manhattan -> Not Buy
Minkowski -> Buy
Hamming -> Buy

Most suitable distance measure:
For the complete dataset, Manhattan distance can be useful because it handles mixed numerical and binary features and is less affected by individual large
However, the numerical features should ideally be normalized before applying KNN because Browsing Time and Items Viewed have different scales from the bina

```

Question 3: Medical Risk Classification

```
1 import math
2 from collections import Counter
3 patients = [
4     ["P1", 25, 90, 0, 0, 0, "Low Risk"],
5     ["P2", 30, 95, 0, 1, 0, "Low Risk"],
6     ["P3", 50, 160, 1, 1, 1, "High Risk"],
7     ["P4", 55, 170, 1, 1, 1, "High Risk"],
8     ["P5", 40, 120, 0, 1, 1, "Low Risk"],
9     ["P6", 60, 180, 1, 1, 1, "High Risk"]
10 ]
11 new_patient = [45, 150, 1, 1, 0]
12
13 K = 3
14 numerical_features = [
15     "Age",
16     "Blood Sugar Level"
17 ]
18
19 binary_features = [
20     "Fever",
21     "Cough",
22     "Fatigue"
23 ]
24
25 print("-" * 65)
26 print("(a) FEATURES")
27 print("-" * 65)
28
29 print("Numerical Features:")
30 for feature in numerical_features:
31     print("-", feature)
32
33 print("\nBinary Features:")
34 for feature in binary_features:
35     print("-", feature)
36
37 X = [patient[1:6] for patient in patients]
38 y = [patient[6] for patient in patients]
39
40 def euclidean_distance(a, b):
41     total = 0
42
43     for i in range(len(a)):
44         total += (a[i] - b[i]) ** 2
45
46     return math.sqrt(total)
```

```
54     return total
55
56 def minkowski_distance(a, b, p=3):
57     total = 0
58
59     for i in range(len(a)):
60         total += abs(a[i] - b[i]) ** p
61
62     return total ** (1 / p)
63
64
65 def hamming_distance(a, b):
66     distance = 0
67
68     for i in range(len(a)):
69         if a[i] != b[i]:
70             distance += 1
71
72     return distance
73
74 euclidean_distances = []
75
76 for i in range(len(X)):
77     distance = euclidean_distance(
78         new_patient,
79         X[i]
80     )
81
82     euclidean_distances.append(distance)
83
84 print("\n" + "-" * 65)
85 print("(b) EUCLIDEAN DISTANCE")
86 print("-" * 65)
87
88 for i in range(len(X)):
89     print(
90         patients[i][0],
91         "->",
92         round(euclidean_distances[i], 4)
93     )
94
95 manhattan_distances = []
96
97 for i in range(len(X)):
98     distance = manhattan_distance(
99         new_patient,
100         X[i]
101     )
```

```
146
147     distance = hamming_distance(
148         new_binary,
149         patient_binary
150     )
151
152     hamming_distances.append(distance)
153
154 print("\n" + "-" * 65)
155 print("(e) HAMMING DISTANCE")
156 print("-" * 65)
157
158 print("Hamming distance uses only binary features.")
159
160 for i in range(len(X)):
161     print(
162         patients[i][0],
163         "->",
164         hamming_distances[i]
165     )
166
167 def get_nearest_neighbours(distances, k=3):
168     sorted_indices = sorted(
169         range(len(distances)),
170         key=lambda i: distances[i]
171     )
172
173     return sorted_indices[:k]
174
175 def predict_class(distances, k=3):
176     nearest_indices = get_nearest_neighbours(
177         distances,
178         k
179     )
180
181     nearest_labels = [
182         y[i] for i in nearest_indices
183     ]
184
185     votes = Counter(nearest_labels)
186
187     prediction = votes.most_common(1)[0][0]
188
189     return nearest_indices, nearest_labels, prediction
190
191 indices, labels, prediction = predict_class(
192     euclidean_distances,
```

```
220
221 for i in indices:
222     print(
223         patients[i][0],
224         "Distance =",
225         round(manhattan_distances[i], 4),
226         "Class =",
227         y[i]
228     )
229
230 print("Neighbour Classes:", labels)
231 print("Prediction:", prediction)
232
233 indices, labels, prediction = predict_class(
234     minkowski_distances,
235     K
236 )
237
238 print("\n" + "-" * 65)
239 print("MINKOWSKI p = 3 - K = 3")
240 print("-" * 65)
241
242 print("3 Nearest Neighbours:")
243
244 for i in indices:
245     print(
246         patients[i][0],
247         "Distance =",
248         round(minkowski_distances[i], 4),
249         "Class =",
250         y[i]
251     )
252
253 print("Neighbour Classes:", labels)
254 print("Prediction:", prediction)
255
256 indices, labels, prediction = predict_class(
257     hamming_distances,
258     K
259 )
260
261 print("\n" + "-" * 65)
262 print("HAMMING - K = 3")
263 print("-" * 65)
264
265 print("3 Nearest Neighbours:")
266
267 for i in indices:
268     print(
269         patients[i][0],
270         "Distance =",
271         hamming_distances[i],
272         "Class =",
```

Output:

```
=====
(a) FEATURES
=====
Numerical Features:
- Age
- Blood Sugar Level

Binary Features:
- Fever
- Cough
- Fatigue

=====
(b) EUCLIDEAN DISTANCE
=====
P1 -> 63.2614
P2 -> 57.0175
P3 -> 11.225
P4 -> 22.383
P5 -> 30.4467
P6 -> 33.5559

=====
(c) MANHATTAN DISTANCE
=====
P1 -> 82
P2 -> 71
P3 -> 16
P4 -> 31
```

```
=====
(d) MINKOWSKI DISTANCE (p = 3)
=====
P1 -> 60.732
P2 -> 55.3695
P3 -> 10.4035
P4 -> 20.8016
P5 -> 30.047
P6 -> 31.2016

=====
(e) HAMMING DISTANCE
=====
Hamming distance uses only binary features.
P1 -> 2
P2 -> 1
P3 -> 1
P4 -> 1
P5 -> 2
P6 -> 1

=====
(f) & (g) EUCLIDEAN - K = 3
=====
3 Nearest Neighbours:
P3 Distance = 11.225 Class = High Risk
P4 Distance = 22.383 Class = High Risk
P5 Distance = 30.4467 Class = Low Risk
Neighbour Classes: ['High Risk', 'High Risk', 'Low Risk']
```

```
=====
MANHATTAN - K = 3
=====
3 Nearest Neighbours:
P3 Distance = 16 Class = High Risk
P4 Distance = 31 Class = High Risk
P5 Distance = 37 Class = Low Risk
Neighbour Classes: ['High Risk', 'High Risk', 'Low Risk']
Prediction: High Risk

=====
MINKOWSKI p = 3 - K = 3
=====
3 Nearest Neighbours:
P3 Distance = 10.4035 Class = High Risk
P4 Distance = 20.8016 Class = High Risk
P5 Distance = 30.047 Class = Low Risk
Neighbour Classes: ['High Risk', 'High Risk', 'Low Risk']
Prediction: High Risk

=====
HAMMING - K = 3
=====
3 Nearest Neighbours:
P2 Distance = 1 Class = Low Risk
P3 Distance = 1 Class = High Risk
P4 Distance = 1 Class = High Risk
Neighbour Classes: ['Low Risk', 'High Risk', 'High Risk']
Prediction: High Risk
```

```
=====
(h) FINAL COMPARISON
=====

Euclidean      -> High Risk
Manhattan      -> High Risk
Minkowski (p=3) -> High Risk
Hamming        -> High Risk

Most suitable distance measure:
Manhattan distance can be suitable for this mixed dataset because it handles numerical and binary features and is less sensitive to large individual differences.

In a real medical KNN system, numerical features such as Age and Blood Sugar Level should be normalized before calculating distance.
```


Question 4: Fruit Classification :

```
1 import math
2 from collections import Counter
3 fruits = [
4     ["F1", 150, 7, 1, 0, 1, "Apple"],
5     ["F2", 160, 8, 1, 0, 1, "Apple"],
6     ["F3", 180, 9, 0, 1, 0, "Orange"],
7     ["F4", 200, 10, 0, 1, 0, "Orange"],
8     ["F5", 155, 7.5, 1, 0, 1, "Apple"],
9     ["F6", 190, 9.5, 0, 1, 0, "Orange"]
10 ]
11 new_fruit = [170, 8.5, 1, 0, 1]
12 K = 3
13 numerical_features = [
14     "Weight",
15     "Diameter"
16 ]
17 binary_features = [
18     "Red Color",
19     "Citrus Smell",
20     "Smooth Skin"
21 ]
22 print("\n" + "=" * 65)
23 print("(a) FEATURES")
24 print("=" * 65)
25 print("Numerical Features:")
26 for feature in numerical_features:
27     print("-", feature)
28 print("\nBinary Features:")
29 for feature in binary_features:
30     print("-", feature)
31 X = [fruit[1:6] for fruit in fruits]
32 y = [fruit[6] for fruit in fruits]
33 def euclidean_distance(a, b):
34     total = 0
35     for i in range(len(a)):
36         total += (a[i] - b[i]) ** 2
37     return math.sqrt(total)
38 def manhattan_distance(a, b):
39     total = 0
40     for i in range(len(a)):
41         total += abs(a[i] - b[i])
42     return total
43 def minkowski_distance(a, b, p=3):
44     total = 0
45     for i in range(len(a)):
46         total += abs(a[i] - b[i]) ** p
47     return total ** (1 / p)
48 def hamming_distance(a, b):
```

```
47     return total ** (1 / p)
48 def hamming_distance(a, b):
49     distance = 0
50     for i in range(len(a)):
51         if a[i] != b[i]:
52             distance += 1
53     return distance
54 euclidean_distances = []
55 for i in range(len(X)):
56     distance = euclidean_distance(
57         new_fruit,
58         X[i]
59     )
60     euclidean_distances.append(distance)
61 print("\n" + "=" * 65)
62 print("(b) EUCLIDEAN DISTANCE")
63 print("=" * 65)
64 for i in range(len(X)):
65     print(
66         fruits[i][0],
67         "->",
68         round(euclidean_distances[i], 4)
69     )
70 manhattan_distances = []
71 for i in range(len(X)):
72     distance = manhattan_distance(
73         new_fruit,
74         X[i]
75     )
76     manhattan_distances.append(distance)
77 print("\n" + "=" * 65)
78 print("(c) MANHATTAN DISTANCE")
79 print("=" * 65)
80 for i in range(len(X)):
81     print(
82         fruits[i][0],
83         "->",
84         round(manhattan_distances[i], 4)
85     )
86 minkowski_distances = []
87 for i in range(len(X)):
88     distance = minkowski_distance(
89         new_fruit,
90         X[i],
91         p=3
92     )
93     minkowski_distances.append(distance)
```

```
113 print("\n" + "=" * 65)
114 print("(e) HAMMING DISTANCE")
115 print("=" * 65)
116 print("Hamming distance uses only binary features.")
117 for i in range(len(X)):
118     print(
119         fruits[i][0],
120         "->",
121         hamming_distances[i]
122     )
123 def get_nearest_neighbours(distances, k=3):
124     sorted_indices = sorted(
125         range(len(distances)),
126         key=lambda i: distances[i]
127     )
128     return sorted_indices[:k]
129 def predict_class(distances, k=3):
130     nearest_indices = get_nearest_neighbours(
131         distances,
132         k
133     )
134     nearest_labels = [
135         y[i] for i in nearest_indices
136     ]
137     votes = Counter(nearest_labels)
138     prediction = votes.most_common(1)[0][0]
139     indices, labels, prediction = predict_class(
140         euclidean_distances,
141         K
142     )
143     print("\n" + "=" * 65)
144     print("(f) & (g) EUCLIDEAN - K = 3")
145     print("=" * 65)
146     print("3 Nearest Neighbours:")
147     for i in indices:
148         print(
149             fruits[i][0],
150             "Distance =",
151             round(euclidean_distances[i], 4),
152             "Class =",
153             y[i]
154         )
```

```
191         fruits[i][0],
192         "Distance =",
193         round(minkowski_distances[i], 4),
194         "Class =",
195         y[i]
196     )
197 print("Neighbour Classes:", labels)
198 print("Prediction:", prediction)
199 indices, labels, prediction = predict_class(
200     hamming_distances,
201     K
202 )
203 print("\n" + "=" * 65)
204 print("HAMMING - K = 3")
205 print("=" * 65)
206 print("3 Nearest Neighbours:")
207 for i in indices:
208     print(
209         fruits[i][0],
210         "Distance =",
211         hamming_distances[i],
212         "Class =",
213         y[i]
214     )
215 print("Neighbour Classes:", labels)
216 print("Prediction:", prediction)
217 print("\n" + "=" * 65)
218 print("(h) FINAL COMPARISON")
219 print("=" * 65)
220 print("""
221 Euclidean    -> Apple
222 Manhattan    -> Apple
223 Minkowski (p=3) -> Apple
224 Hamming      -> Apple
225 """)
226 print("Most suitable distance measure:")
227 print(
228     "For the complete dataset, Manhattan distance can be suitable "
229     "because the dataset contains both numerical and binary features."
230 )
231 print(
232     "\nFor a real KNN application, Weight and Diameter should be "
233     "normalized before calculating Euclidean, Manhattan, or "
234     "Minkowski distances."
235 )
```

Output:

```
=====
(a) FEATURES
=====
Numerical Features:
- Weight
- Diameter

Binary Features:
- Red Color
- Citrus Smell
- Smooth Skin

=====
(b) EUCLIDEAN DISTANCE
=====
F1 -> 20.0562
F2 -> 10.0125
F3 -> 10.1612
F4 -> 30.0874
F5 -> 15.0333
F6 -> 20.0998

=====
(c) MANHATTAN DISTANCE
=====
F1 -> 21.5
F2 -> 10.5
F3 -> 13.5
F4 -> 34.5
F5 -> 16.0
```

```
=====
(d) MINKOWSKI DISTANCE (p = 3)
=====
F1 -> 20.0028
F2 -> 10.0004
F3 -> 10.0104
F4 -> 30.0024
F5 -> 15.0015
F6 -> 20.0033

=====
(e) HAMMING DISTANCE
=====
Hamming distance uses only binary features.
F1 -> 0
F2 -> 0
F3 -> 3
F4 -> 3
F5 -> 0
F6 -> 3

=====
(f) & (g) EUCLIDEAN - K = 3
=====
3 Nearest Neighbours:
F2 Distance = 10.0125 Class = Apple
F3 Distance = 10.1612 Class = Orange
F5 Distance = 15.0333 Class = Apple
Neighbour Classes: ['Apple', 'Orange', 'Apple']
Prediction: Apple
```

```
=====
MANHATTAN - K = 3
=====
3 Nearest Neighbours:
F2 Distance = 10.5 Class = Apple
F3 Distance = 13.5 Class = Orange
F5 Distance = 16.0 Class = Apple
Neighbour Classes: ['Apple', 'Orange', 'Apple']
Prediction: Apple

=====
MINKOWSKI p = 3 - K = 3
=====
3 Nearest Neighbours:
F2 Distance = 10.0004 Class = Apple
F3 Distance = 10.0104 Class = Orange
F5 Distance = 15.0015 Class = Apple
Neighbour Classes: ['Apple', 'Orange', 'Apple']
Prediction: Apple

=====
HAMMING - K = 3
=====
3 Nearest Neighbours:
F1 Distance = 0 Class = Apple
F2 Distance = 0 Class = Apple
F5 Distance = 0 Class = Apple
Neighbour Classes: ['Apple', 'Apple', 'Apple']
Prediction: Apple
```

```
=====
(h) FINAL COMPARISON
=====

Euclidean      -> Apple
Manhattan      -> Apple
Minkowski (p=3) -> Apple
Hamming        -> Apple

Most suitable distance measure:
For the complete dataset, Manhattan distance can be suitable because the dataset contains both numerical and binary features.

For a real KNN application, Weight and Diameter should be normalized before calculating Euclidean, Manhattan, or Minkowski distances.
```

5. Question 5: House Price Category Prediction

```
1 import math
2 from collections import Counter
3 houses = [
4     ["H1", 800, 2, 0, 0, 0, "Low Price"],
5     ["H2", 1000, 2, 1, 0, 0, "Low Price"],
6     ["H3", 1500, 3, 1, 1, 1, "High Price"],
7     ["H4", 1800, 4, 1, 1, 1, "High Price"],
8     ["H5", 1200, 3, 0, 1, 0, "Low Price"],
9     ["H6", 2000, 4, 1, 1, 1, "High Price"]
10 ]
11
12 # New house
13 new_house = [1400, 3, 1, 1, 0]
14
15 K = 3
16
17 numerical_features = [
18     "Area (sq.ft)",
19     "Bedrooms"
20 ]
21
22 binary_features = [
23     "Parking Available",
24     "Near Main Road",
25     "Garden Available"
26 ]
27
28 print("-" * 70)
29 print("(a) FEATURES")
30 print("-" * 70)
31
32 print("Numerical Features:")
33 for feature in numerical_features:
34     print("-", feature)
35
36 print("\nBinary Features:")
37 for feature in binary_features:
38     print("-", feature)
39
40 X = [house[1:6] for house in houses]
41 y = [house[6] for house in houses]
42
43 def euclidean_distance(a, b):
44     total = 0
45
46     for i in range(len(a)):
47         total += (a[i] - b[i]) ** 2
48
```

```
102     manhattan_distances.append(distance)
103
104 print("\n" + "-" * 70)
105 print("(c) MANHATTAN DISTANCE")
106 print("-" * 70)
107
108 for i in range(len(X)):
109     print(
110         houses[i][0],
111         "->",
112         round(manhattan_distances[i], 4)
113     )
114     manhattan_distances = []
115
116 for i in range(len(X)):
117
118     distance = minkowski_distance(
119         new_house,
120         X[i],
121         p=3
122     )
123
124     minkowski_distances.append(distance)
125
126 print("\n" + "-" * 70)
127 print("(d) MINKOWSKI DISTANCE (p = 3)")
128 print("-" * 70)
129
130 for i in range(len(X)):
131     print(
132         houses[i][0],
133         "->",
134         round(minkowski_distances[i], 4)
135     )
136
137 new_binary = new_house[2:5]
138
139 hamming_distances = []
140
141 for i in range(len(X)):
142
143     house_binary = X[i][2:5]
144
145     distance = hamming_distance(
146         new_binary,
147         house_binary
148     )
149
150     hamming_distances.append(distance)

```

```
252 print("Prediction:", prediction)
253 indices, labels, prediction = predict_class(
254     hamming_distances,
255     K
256 )
257
258 print("\n" + "-" * 70)
259 print("HAMMING - K = 3")
260 print("-" * 70)
261
262 print("3 Nearest Neighbours:")
263
264 for i in indices:
265     print(
266         houses[i][0],
267         "Distance =",
268         hamming_distances[i],
269         "Category =",
270         y[i]
271     )
272
273 print("Neighbour Categories:", labels)
274 print("Prediction:", prediction)
275
276 print("\n" + "-" * 70)
277 print("(h) FINAL COMPARISON")
278 print("-" * 70)
279
280 print("""
281 Euclidean    -> High Price
282 Manhattan    -> High Price
283 Minkowski (p=3) -> High Price
284 Hamming      -> High Price
285 """)
286
287 print("Most suitable distance measure:")
288 print(
289     "For this mixed dataset, Manhattan distance is a suitable "
290     "general-purpose measure because it handles numerical and "
291     "binary features and is less sensitive to large individual "
292     "differences."
293 )
294
295 print(
296     "\nIn a real KNN model, Area and Bedrooms should be normalized "
297     "before calculating the distances because Area has a much "
298     "larger numerical scale."
299 )

```

Output:

```
=====
Numerical Features:
- Area (sq.ft)
- Bedrooms

Binary Features:
- Parking Available
- Near Main Road
- Garden Available

=====
(b) EUCLIDEAN DISTANCE
=====
H1 -> 600.0025
H2 -> 400.0025
H3 -> 100.005
H4 -> 400.0025
H5 -> 200.0025
H6 -> 600.0017

=====
(c) MANHATTAN DISTANCE
=====
H1 -> 603
H2 -> 402
H3 -> 101
H4 -> 402
H5 -> 201
H6 -> 602
```

```
=====
(d) MINKOWSKI DISTANCE (p = 3)
=====
H1 -> 600.0
H2 -> 400.0
H3 -> 100.0
H4 -> 400.0
H5 -> 200.0
H6 -> 600.0

=====
(e) HAMMING DISTANCE
=====
Hamming distance uses only binary features.
H1 -> 2
H2 -> 1
H3 -> 1
H4 -> 1
H5 -> 1
H6 -> 1

=====
(f) & (g) EUCLIDEAN - K = 3
=====
3 Nearest Neighbours:
H3 Distance = 100.005 Category = High Price
H5 Distance = 200.0025 Category = Low Price
H2 Distance = 400.0025 Category = Low Price
Neighbour Categories: ['High Price', 'Low Price', 'Low Price']
```

```
=====
MANHATTAN - K = 3
=====
3 Nearest Neighbours:
H3 Distance = 101 Category = High Price
H5 Distance = 201 Category = Low Price
H2 Distance = 402 Category = Low Price
Neighbour Categories: ['High Price', 'Low Price', 'Low Price']
Prediction: Low Price

=====
MINKOWSKI p = 3 - K = 3
=====
3 Nearest Neighbours:
H3 Distance = 100.0 Category = High Price
H5 Distance = 200.0 Category = Low Price
H2 Distance = 400.0 Category = Low Price
Neighbour Categories: ['High Price', 'Low Price', 'Low Price']
Prediction: Low Price

=====
HAMMING - K = 3
=====
3 Nearest Neighbours:
H2 Distance = 1 Category = Low Price
H3 Distance = 1 Category = High Price
H4 Distance = 1 Category = High Price
Neighbour Categories: ['Low Price', 'High Price', 'High Price']
Prediction: High Price
```

```
=====
(h) FINAL COMPARISON
=====

Euclidean      -> High Price
Manhattan      -> High Price
Minkowski (p=3) -> High Price
Hamming        -> High Price

Most suitable distance measure:
For this mixed dataset, Manhattan distance is a suitable general-purpose measure because it handles numerical and binary features and is less sensitive to outliers.

In a real KNN model, Area and Bedrooms should be normalized before calculating the distances because Area has a much larger numerical scale.
```

Question 6: Loan Approval Prediction

```
1 import math
2 from collections import Counter
3 applicants = [
4     ["A1", 25000, 580, 0, 1, 1, "Rejected"],
5     ["A2", 30000, 600, 0, 1, 0, "Rejected"],
6     ["A3", 60000, 750, 1, 0, 0, "Approved"],
7     ["A4", 70000, 800, 1, 0, 0, "Approved"],
8     ["A5", 40000, 650, 0, 1, 0, "Rejected"],
9     ["A6", 80000, 820, 1, 0, 0, "Approved"]
10 ]
11 new_applicant = [55000, 700, 1, 0, 0]
12
13 K = 3
14 numerical_features = [
15     "Monthly Income",
16     "Credit Score"
17 ]
18
19 binary_features = [
20     "Govt Job",
21     "Existing Loan",
22     "Past Default"
23 ]
24
25 print("\n" + "-" * 70)
26 print("(a) FEATURES")
27 print("\n" + "-" * 70)
28
29 print("Numerical Features:")
30 for feature in numerical_features:
31     print("-", feature)
32
33 print("\nBinary Features:")
34 for feature in binary_features:
35     print("-", feature)
36
37 X = [applicant[1:6] for applicant in applicants]
38 y = [applicant[6] for applicant in applicants]
39
40 def euclidean_distance(a, b):
41     total = 0
42     for i in range(len(a)):
43         total += (a[i] - b[i]) ** 2
44     return math.sqrt(total)
45
46 def manhattan_distance(a, b):
47     total = 0
48     for i in range(len(a)):
```

```
101
102 print("\n" + "-" * 70)
103 print("(c) MANHATTAN DISTANCE")
104 print("\n" + "-" * 70)
105
106 for i in range(len(X)):
107     print(
108         applicants[i][0],
109         "-",
110         round(manhattan_distances[i], 4)
111     )
112
113 minkowski_distances = []
114
115 for i in range(len(X)):
116
117     distance = minkowski_distance(
118         new_applicant,
119         X[i],
120         p=3
121     )
122
123     minkowski_distances.append(distance)
124
125 print("\n" + "-" * 70)
126 print("(d) MINKOWSKI DISTANCE (p = 3)")
127 print("\n" + "-" * 70)
128
129 for i in range(len(X)):
130     print(
131         applicants[i][0],
132         "-",
133         round(minkowski_distances[i], 4)
134     )
135
136 new_binary = new_applicant[2:5]
137
138 hamming_distances = []
139
140 for i in range(len(X)):
141
142     applicant_binary = X[i][2:5]
143
144     distance = hamming_distance(
145         new_binary,
146         applicant_binary
147     )
148
```

```
164 def get_nearest_neighbours(distances, k=3):
165     sorted_indices = sorted(
166         range(len(distances)),
167         key=lambda i: distances[i]
168     )
169
170     return sorted_indices[:k]
171
172 def predict_class(distances, k=3):
173
174     nearest_indices = get_nearest_neighbours(
175         distances,
176         k
177     )
178
179     nearest_labels = [
180         y[i] for i in nearest_indices
181     ]
182
183     votes = Counter(nearest_labels)
184
185     prediction = votes.most_common(1)[0][0]
186
187     return nearest_indices, nearest_labels, prediction
188
189 indices, labels, prediction = predict_class(
190     euclidean_distances,
191     K
192 )
193
194 print("\n" + "-" * 70)
195 print("(f) & (g) EUCLIDEAN - K = 3")
196 print("\n" + "-" * 70)
197
198 print("3 Nearest Neighbours:")
199
200 for i in indices:
201     print(
202         applicants[i][0],
203         "Distance =",
204         round(euclidean_distances[i], 4),
205         "Class =",
206         y[i]
207     )
208
209 print("Neighbour Classes:", labels)
210 print("Prediction:", prediction)
211 indices, labels, prediction = predict_class(
212     manhattan_distances,
```

```
253 print("Neighbour Classes:", labels)
254 print("Prediction:", prediction)
255 indices, labels, prediction = predict_class(
256     hamming_distances,
257     K
258 )
259
260 print("\n" + "-" * 70)
261 print("HAMMING - K = 3")
262 print("\n" + "-" * 70)
263
264 print("3 Nearest Neighbours:")
265
266 for i in indices:
267     print(
268         applicants[i][0],
269         "Distance =",
270         hamming_distances[i],
271         "Class =",
272         y[i]
273     )
274
275 print("Neighbour Classes:", labels)
276 print("Prediction:", prediction)
277 print("\n" + "-" * 70)
278 print("(h) FINAL COMPARISON")
279 print("\n" + "-" * 70)
280
281 print("""
282 Euclidean    -> Approved
283 Manhattan    -> Approved
284 Minkowski (p=3) -> Approved
285 Hamming      -> Approved
286 """)
287
288 print("Most suitable distance measure:")
289
290 print(
291     "For this mixed dataset, Manhattan distance is a reasonable "
292     "general-purpose choice because the dataset contains both "
293     "numerical and binary features."
294 )
295
296 print(
297     "\nHowever, Monthly Income and Credit Score should be normalized "
298     "before applying KNN in a real application because Monthly Income "
299     "has a much larger numerical scale."
300 )
```

Output:

```
Numerical Features:
- Monthly Income
- Credit Score
```

```
Binary Features:
- Govt Job
- Existing Loan
- Past Default
```

(b) EUCLIDEAN DISTANCE

```
A1 -> 30000.24
A2 -> 25000.2
A3 -> 5000.25
A4 -> 15000.3333
A5 -> 15000.0834
A6 -> 25000.288
```

(c) MANHATTAN DISTANCE

```
A1 -> 30123
A2 -> 25102
A3 -> 5050
A4 -> 15100
A5 -> 15052
A6 -> 25120
```

(d) MINKOWSKI DISTANCE (p = 3)

```
A1 -> 30000.0006
A2 -> 25000.0005
A3 -> 5000.0017
A4 -> 15000.0015
A5 -> 15000.0002
A6 -> 25000.0009
```

(e) HAMMING DISTANCE

Hamming distance uses only binary features.

```
A1 -> 3
A2 -> 2
A3 -> 0
A4 -> 0
A5 -> 2
A6 -> 0
```

(f) & (g) EUCLIDEAN - K = 3

3 Nearest Neighbours:

```
A3 Distance = 5000.25 Class = Approved
A5 Distance = 15000.0834 Class = Rejected
A4 Distance = 15000.3333 Class = Approved
Neighbour Classes: ['Approved', 'Rejected', 'Approved']
Prediction: Approved
```

MANHATTAN - K = 3

3 Nearest Neighbours:

```
A3 Distance = 5050 Class = Approved
A5 Distance = 15052 Class = Rejected
A4 Distance = 15100 Class = Approved
Neighbour Classes: ['Approved', 'Rejected', 'Approved']
Prediction: Approved
```

MINKOWSKI p = 3 - K = 3

3 Nearest Neighbours:

```
A3 Distance = 5000.0017 Class = Approved
A5 Distance = 15000.0002 Class = Rejected
A4 Distance = 15000.0015 Class = Approved
Neighbour Classes: ['Approved', 'Rejected', 'Approved']
Prediction: Approved
```

HAMMING - K = 3

3 Nearest Neighbours:

```
A3 Distance = 0 Class = Approved
A4 Distance = 0 Class = Approved
A6 Distance = 0 Class = Approved
Neighbour Classes: ['Approved', 'Approved', 'Approved']
Prediction: Approved
```

(h) FINAL COMPARISON

```
Euclidean    -> Approved
Manhattan    -> Approved
Minkowski (p=3) -> Approved
Hamming      -> Approved
```

Most suitable distance measure:

For this mixed dataset, Manhattan distance is a reasonable general-purpose choice because the dataset contains both numerical and binary features.

However, Monthly Income and Credit Score should be normalized before applying KNN in a real application because Monthly Income has a much larger numerical

Question 7: Movie Preference Classification

```
photo.py > ...
1 import math
2 from collections import Counter
3 users = [
4     ["U1", 5, 1, 1, 0, 0, "Action Lover"],
5     ["U2", 4, 2, 1, 0, 0, "Action Lover"],
6     ["U3", 1, 5, 0, 1, 1, "Comedy Lover"],
7     ["U4", 2, 4, 0, 1, 1, "Comedy Lover"],
8     ["U5", 5, 2, 1, 0, 0, "Action Lover"],
9     ["U6", 1, 4, 0, 1, 1, "Comedy Lover"]
10 ]
11 new_user = [3, 3, 1, 1, 0]
12
13 K = 3
14
15 numerical_features = [
16     "Action Rating",
17     "Comedy Rating"
18 ]
19
20 binary_features = [
21     "Watched Superhero Movie",
22     "Watched Stand-up Show",
23     "Watched Sitcom"
24 ]
25
26 print("=" * 70)
27 print("(a) FEATURES")
28 print("=" * 70)
29
30 print("Numerical Features:")
31 for feature in numerical_features:
32     print("-", feature)
33
34 print("\nBinary Features:")
35 for feature in binary_features:
36     print("-", feature)
37
38 X = [user[1:6] for user in users]
39 y = [user[6] for user in users]
40
41 def euclidean_distance(a, b):
42     total = 0
43
44     for i in range(len(a)):
45         total += (a[i] - b[i]) ** 2
46
47     return math.sqrt(total)
48
```

```
41 def euclidean_distance(a, b):
42     total = 0
43
44     for i in range(len(a)):
45         total += (a[i] - b[i]) ** 2
46
47     return math.sqrt(total)
48
49 def manhattan_distance(a, b):
50     total = 0
51
52     for i in range(len(a)):
53         total += abs(a[i] - b[i])
54
55     return total
56
57 def minkowski_distance(a, b, p=3):
58     total = 0
59
60     for i in range(len(a)):
61         total += abs(a[i] - b[i]) ** p
62
63     return total ** (1 / p)
64
65 def hamming_distance(a, b):
66     distance = 0
67
68     for i in range(len(a)):
69         if a[i] != b[i]:
70             distance += 1
71
72     return distance
73
74 euclidean_distances = []
75
76 for i in range(len(X)):
77     distance = euclidean_distance(
78         new_user,
79         X[i]
80     )
81     euclidean_distances.append(distance)
82
83 print("\n" + "=" * 70)
84 print("(b) EUCLIDEAN DISTANCE")
85 print("=" * 70)
86
87 for i in range(len(X)):
88     print(

```

```
260
261 print("\n" + "=" * 70)
262 print("HAMMING - K = 3")
263 print("=" * 70)
264
265 print("3 Nearest Neighbours:")
266
267 for i in indices:
268     print(
269         users[i][0],
270         "Distance =",
271         hamming_distances[i],
272         "Class =",
273         y[i]
274     )
275
276 print("Neighbour Classes:", labels)
277 print("Prediction:", prediction)
278 print("\n" + "=" * 70)
279 print("(h) FINAL COMPARISON")
280 print("=" * 70)
281
282 print("""
283 Euclidean    -> Action Lover
284 Manhattan    -> Action Lover
285 Minkowski (p=3) -> Action Lover
286 Hamming      -> Action Lover
287 """)
288
289 print("Most suitable distance measure:")
290
291 print(
292     "For this mixed dataset, Manhattan distance is a reasonable "
293     "general-purpose choice because it handles numerical and "
294     "binary features."
295 )
296
297 print(
298     "\nHamming distance is useful when the focus is specifically "
299     "on the binary viewing behaviour."
300 )
301
302 print(
303     "\nFor a real KNN model, numerical ratings should be normalized "
304     "before calculating Euclidean, Manhattan, or Minkowski distances."
305 )

```

Output:

```
=====
(a) FEATURES
=====
Numerical Features:
- Action Rating
- Comedy Rating

Binary Features:
- Watched Superhero Movie
- Watched Stand-up Show
- Watched Sitcom

=====
(b) EUCLIDEAN DISTANCE
=====
U1 -> 3.0
U2 -> 1.7321
U3 -> 3.1623
U4 -> 2.0
U5 -> 2.4495
U6 -> 2.6458

=====
(c) MANHATTAN DISTANCE
=====
U1 -> 5
U2 -> 3
U3 -> 6
U4 -> 4
U5 -> 4
```

```
=====
(d) MINKOWSKI DISTANCE (p = 3)
=====
U1 -> 2.5713
U2 -> 1.4422
U3 -> 2.6207
U4 -> 1.5874
U5 -> 2.1544
U6 -> 2.224

=====
(e) HAMMING DISTANCE
=====
Hamming distance uses only binary features.
U1 -> 1
U2 -> 1
U3 -> 2
U4 -> 2
U5 -> 1
U6 -> 2

=====
(f) & (g) EUCLIDEAN - K = 3
=====
3 Nearest Neighbours:
U2 Distance = 1.7321 Class = Action Lover
U4 Distance = 2.0 Class = Comedy Lover
U5 Distance = 2.4495 Class = Action Lover
Neighbour Classes: ['Action Lover', 'Comedy Lover', 'Action Lover']
Prediction: Action Lover
```

```
MANHATTAN - K = 3
=====
3 Nearest Neighbours:
U2 Distance = 3 Class = Action Lover
U4 Distance = 4 Class = Comedy Lover
U5 Distance = 4 Class = Action Lover
Neighbour Classes: ['Action Lover', 'Comedy Lover', 'Action Lover']
Prediction: Action Lover

=====
MINKOWSKI p = 3 - K = 3
=====
3 Nearest Neighbours:
U2 Distance = 1.4422 Class = Action Lover
U4 Distance = 1.5874 Class = Comedy Lover
U5 Distance = 2.1544 Class = Action Lover
Neighbour Classes: ['Action Lover', 'Comedy Lover', 'Action Lover']
Prediction: Action Lover

=====
HAMMING - K = 3
=====
3 Nearest Neighbours:
U1 Distance = 1 Class = Action Lover
U2 Distance = 1 Class = Action Lover
U5 Distance = 1 Class = Action Lover
Neighbour Classes: ['Action Lover', 'Action Lover', 'Action Lover']
Prediction: Action Lover
```


Question 8: Crop Disease Detection

```
import math
from collections import Counter

plants = [
    ["L1", 2, 80, 0, 0, 0, "Healthy"],
    ["L2", 3, 75, 0, 0, 0, "Healthy"],
    ["L3", 12, 40, 1, 1, 1, "Diseased"],
    ["L4", 15, 35, 1, 1, 1, "Diseased"],
    ["L5", 5, 70, 0, 0, 1, "Healthy"],
    ["L6", 14, 38, 1, 1, 1, "Diseased"]
]

new_plant = [10, 50, 1, 1, 0]
K = 3

numerical_features = [
    "Leaf Spot Count",
    "Leaf Color Index"
]

binary_features = [
    "Yellow Leaves",
    "White Fungus",
    "Wilting"
]

print("=" * 70)
print("(a) FEATURES")
print("=" * 70)
print("Numerical Features:")
for feature in numerical_features:
    print("-", feature)

print("\nBinary Features:")
for feature in binary_features:
    print("-", feature)

X = [plant[1:6] for plant in plants]
y = [plant[6] for plant in plants]

def euclidean_distance(a, b):
    total = 0

    for i in range(len(a)):
        total += (a[i] - b[i]) ** 2

    return math.sqrt(total)
```

```
    plants[i][0],
    "Distance =",
    hamming_distances[i],
    "Class =",
    y[i]
)

print("Neighbour Classes:", labels)
print("Prediction:", prediction)

# =====
# (h) FINAL COMPARISON
# =====

print("\n" + "=" * 70)
print("(h) FINAL COMPARISON")
print("=" * 70)

print("""
Euclidean      -> Diseased
Manhattan      -> Diseased
Minkowski (p=3) -> Diseased
Hamming        -> Diseased
""")

print("Most suitable distance measure:")

print(
    "For this mixed dataset, Manhattan distance is a reasonable "
    "general-purpose choice because it considers both numerical "
    "and binary features."
)

print(
    "\nHamming distance is useful when the focus is specifically "
    "on the binary symptoms such as Yellow Leaves, White Fungus "
    "and Wilting."
)

print(
    "\nFor a real KNN model, numerical features should be normalized "
    "before calculating Euclidean, Manhattan, or Minkowski distances."
)
```

Output:

```
=====
Numerical Features:
- Leaf Spot Count
- Leaf Color Index
```

```
Binary Features:
- Yellow Leaves
- White Fungus
- Wilting
```

```
=====
(b) EUCLIDEAN DISTANCE
```

```
=====
L1 -> 31.0805
L2 -> 26.0
L3 -> 10.247
L4 -> 15.843
L5 -> 20.6882
L6 -> 12.6886
```

```
=====
(c) MANHATTAN DISTANCE
```

```
=====
L1 -> 40
L2 -> 34
L3 -> 13
L4 -> 21
L5 -> 28
L6 -> 17
```

```
=====
(d) MINKOWSKI DISTANCE (p = 3)
```

```
=====
L1 -> 30.1892
L2 -> 25.1827
L3 -> 10.0299
L4 -> 15.1844
L5 -> 20.1061
L6 -> 12.1486
```

```
=====
(e) HAMMING DISTANCE
```

```
=====
Hamming distance uses only binary features.
```

```
L1 -> 2
L2 -> 2
L3 -> 1
L4 -> 1
L5 -> 3
L6 -> 1
```

```
=====
(f) & (g) EUCLIDEAN - K = 3
```

```
=====
3 Nearest Neighbours:
```

```
L3 Distance = 10.247 Class = Diseased
L6 Distance = 12.6886 Class = Diseased
L4 Distance = 15.843 Class = Diseased
Neighbour Classes: ['Diseased', 'Diseased', 'Diseased']
Prediction: Diseased
```

```
L6 Distance = 12.1486 Class = Diseased
L4 Distance = 15.1844 Class = Diseased
Neighbour Classes: ['Diseased', 'Diseased', 'Diseased']
Prediction: Diseased
```

```
=====
HAMMING - K = 3
```

```
=====
3 Nearest Neighbours:
```

```
L3 Distance = 1 Class = Diseased
L4 Distance = 1 Class = Diseased
L6 Distance = 1 Class = Diseased
Neighbour Classes: ['Diseased', 'Diseased', 'Diseased']
Prediction: Diseased
```

```
=====
(h) FINAL COMPARISON
```

```
=====
Euclidean      -> Diseased
Manhattan      -> Diseased
Minkowski (p=3) -> Diseased
Hamming        -> Diseased
```

Most suitable distance measure:

For this mixed dataset, Manhattan distance is a reasonable general-purpose choice because it considers both numerical and binary features.

Hamming distance is useful when the focus is specifically on the binary symptoms such as Yellow Leaves, White Fungus and Wilting.

For a real KNN model, numerical features should be normalized before calculating Euclidean, Manhattan, or Minkowski distances.

Question 9: Network Intrusion Detection

```
import math
from collections import Counter
records = [
    ["N1", 20, 0, 0, 0, 0, "Normal"],
    ["N2", 30, 1, 0, 0, 0, "Normal"],
    ["N3", 100, 5, 1, 1, 1, "Attack"],
    ["N4", 120, 6, 1, 1, 1, "Attack"],
    ["N5", 40, 1, 0, 0, 0, "Normal"],
    ["N6", 110, 7, 1, 1, 1, "Attack"]
]
new_record = [90, 4, 1, 1, 0]

K = 3
numerical_features = [
    "Packets per Second",
    "Failed Login Attempts"
]

binary_features = [
    "Unknown Port Used",
    "Admin Access Tried",
    "Blacklisted IP"
]

print("=" * 70)
print("(a) FEATURES")
print("=" * 70)

print("Numerical Features:")
for feature in numerical_features:
    print("-", feature)

print("\nBinary Features:")
for feature in binary_features:
    print("-", feature)

X = [record[1:6] for record in records]
y = [record[6] for record in records]

def euclidean_distance(a, b):
    total = 0
    for i in range(len(a)):
        total += (a[i] - b[i]) ** 2
    return math.sqrt(total)
```

Output:

```
=====
(a) FEATURES
=====
Numerical Features:
- Packets per Second
- Failed Login Attempts

Binary Features:
- Unknown Port Used
- Admin Access Tried
- Blacklisted IP

=====
(b) EUCLIDEAN DISTANCE
=====
N1 -> 70.1285
N2 -> 60.0916
N3 -> 10.0995
N4 -> 30.0832
N5 -> 50.1099
N6 -> 20.2485

=====
(c) MANHATTAN DISTANCE
=====
N1 -> 76
N2 -> 65
N3 -> 12
N4 -> 33
N5 -> 55
```

```

=====
(e) HAMMING DISTANCE
=====
Hamming distance uses only binary features.
N1 -> 2
N2 -> 2
N3 -> 1
N4 -> 1
N5 -> 2
N6 -> 1

=====
(f) & (g) EUCLIDEAN - K = 3
=====
3 Nearest Neighbours:
N3 Distance = 10.0995 Class = Attack
N6 Distance = 20.2485 Class = Attack
N4 Distance = 30.0832 Class = Attack
Neighbour Classes: ['Attack', 'Attack', 'Attack']
Prediction: Attack

=====
MANHATTAN - K = 3
=====
3 Nearest Neighbours:
N3 Distance = 12 Class = Attack
N6 Distance = 24 Class = Attack
N4 Distance = 33 Class = Attack
Neighbour Classes: ['Attack', 'Attack', 'Attack']
Prediction: Attack

```

```

N6 Distance = 20.0233 Class = Attack
N4 Distance = 30.0033 Class = Attack
Neighbour Classes: ['Attack', 'Attack', 'Attack']
Prediction: Attack

=====
HAMMING - K = 3
=====
3 Nearest Neighbours:
N3 Distance = 1 Class = Attack
N4 Distance = 1 Class = Attack
N6 Distance = 1 Class = Attack
Neighbour Classes: ['Attack', 'Attack', 'Attack']
Prediction: Attack

=====
(h) FINAL COMPARISON
=====

Euclidean      -> Attack
Manhattan      -> Attack
Minkowski (p=3) -> Attack
Hamming        -> Attack

Most suitable distance measure:
For this mixed dataset, Manhattan distance is a reasonable general-purpose choice because it considers both numerical and binary features.

Hamming distance is useful when the focus is specifically on binary security indicators such as Unknown Port, Admin Access, and Blacklisted IP.

For a real KNN intrusion detection system, numerical features should be normalized before calculating distance.

```

Question 10: Employee Attrition Prediction

```
import math
from collections import Counter

employees = [
    ["E1", 2, 8, 0, 1, 0, "Stay"],
    ["E2", 3, 7, 0, 1, 0, "Stay"],
    ["E3", 6, 3, 1, 0, 1, "Leave"],
    ["E4", 7, 2, 1, 0, 1, "Leave"],
    ["E5", 4, 6, 0, 0, 0, "Stay"],
    ["E6", 8, 2, 1, 0, 1, "Leave"]
]

# New employee
new_employee = [5, 4, 1, 0, 1]

K = 3

numerical_features = [
    "Experience in Years",
    "Satisfaction Score"
]

binary_features = [
    "Overtime",
    "Promotion Received",
    "Workplace Conflict"
]

print("=" * 70)
print("(a) FEATURES")
print("=" * 70)

print("Numerical Features:")
for feature in numerical_features:
    print("-", feature)

print("\nBinary Features:")
for feature in binary_features:
    print("-", feature)

X = [employee[1:6] for employee in employees]
y = [employee[6] for employee in employees]
def euclidean_distance(a, b):
    total = 0
```

Output:

```
Numerical Features:
- Experience in Years
- Satisfaction Score

Binary Features:
- Overtime
- Promotion Received
- Workplace Conflict

=====
(b) EUCLIDEAN DISTANCE
=====
E1 -> 5.2915
E2 -> 4.0
E3 -> 1.4142
E4 -> 2.8284
E5 -> 2.6458
E6 -> 3.6056

=====
(c) MANHATTAN DISTANCE
=====
E1 -> 10
E2 -> 8
E3 -> 2
E4 -> 4
E5 -> 5
E6 -> 5
```

```

=====
(d) MINKOWSKI DISTANCE (p = 3)
=====
E1 -> 4.5468
E2 -> 3.362
E3 -> 1.2599
E4 -> 2.5198
E5 -> 2.224
E6 -> 3.2711

=====
(e) HAMMING DISTANCE
=====
Hamming distance uses only binary features.
E1 -> 3
E2 -> 3
E3 -> 0
E4 -> 0
E5 -> 2
E6 -> 0

=====
(f) & (g) EUCLIDEAN - K = 3
=====
3 Nearest Neighbours:
E3 Distance = 1.4142 Class = Leave
E5 Distance = 2.6458 Class = Stay
E4 Distance = 2.8284 Class = Leave
Neighbour Classes: ['Leave', 'Stay', 'Leave']
Prediction: Leave

```

```

=====
MANHATTAN - K = 3
=====
3 Nearest Neighbours:
E3 Distance = 2 Class = Leave
E4 Distance = 4 Class = Leave
E5 Distance = 5 Class = Stay
Neighbour Classes: ['Leave', 'Leave', 'Stay']
Prediction: Leave

=====
MINKOWSKI p = 3 - K = 3
=====
3 Nearest Neighbours:
E3 Distance = 1.2599 Class = Leave
E5 Distance = 2.224 Class = Stay
E4 Distance = 2.5198 Class = Leave
Neighbour Classes: ['Leave', 'Stay', 'Leave']
Prediction: Leave

=====
HAMMING - K = 3
=====
3 Nearest Neighbours:
E3 Distance = 0 Class = Leave
E4 Distance = 0 Class = Leave
E6 Distance = 0 Class = Leave
Neighbour Classes: ['Leave', 'Leave', 'Leave']
Prediction: Leave

```

```

=====
HAMMING - K = 3
=====
3 Nearest Neighbours:
E3 Distance = 0 Class = Leave
E4 Distance = 0 Class = Leave
E6 Distance = 0 Class = Leave
Neighbour Classes: ['Leave', 'Leave', 'Leave']
Prediction: Leave

=====
(h) FINAL COMPARISON
=====

Euclidean      -> Leave
Manhattan      -> Leave
Minkowski (p=3) -> Leave
Hamming        -> Leave

Most suitable distance measure:
For this mixed dataset, Manhattan distance is a reasonable general-purpose choice because it considers both numerical and binary features.

Hamming distance is useful when the focus is specifically on binary workplace factors such as Overtime, Promotion Received, and Workplace Conflict.

For a real KNN model, Experience and Satisfaction Score should be normalized before calculating distance.

```